

# Embedded Systems Architecture

Mohamed Mesbah | IEEE-ZSB RAS-Embedded 2026

---

## 1 Memory Hierarchy & Cache Memory

### Speed vs. Size

- There is an inverse relationship between memory size and speed.
- Registers are the fastest — they reside inside the CPU and are accessed by name, not by address.
- Cache (SRAM) is the next fastest, followed by main memory (RAM), and finally the Hard Disk, which is the slowest but has the largest capacity.

### Cache Mechanism

- Cache acts as a fast intermediary between the CPU and the slower RAM.
- When the CPU requests an address, the cache fetches that address along with a range of subsequent addresses from RAM, saving time for future requests.

### Hits, Misses & Cache Coherence

- **Hit:** requested data is found in the cache. **Miss:** data is not found, and the cache must fetch a new range from RAM.
- In multi-core systems, if one core modifies cached data, other cores may hold outdated copies — this is resolved by cache controllers that communicate to broadcast updates and keep data synchronized.

### Co-processors & Protection Units

- **FPU (Floating Point Unit)** — dedicated hardware for complex math and floating-point operations, offloading the main CPU.
- **MPU (Memory Protection Unit)** — sits between cache and RAM; enforces memory access permissions and prevents different OSes or tasks from crossing into each other's memory, raising a fault on violation.
- **Virtual Memory** — lets large applications (e.g., 8 GB) run on much smaller physical RAM (e.g., 1 GB) by using part of the Hard Disk to simulate extra memory.

### System Buses

- **Data Bus** — bidirectional; transfers data back and forth.
- **Address Bus** — unidirectional; carries addresses from the CPU to components.
- **Control Bus** — carries control signals (e.g., Memory Read/Write), typically *Active Low* (activated at logic 0).

### CPU Architectures: Von Neumann vs. Harvard

- **Von Neumann** — ROM and RAM share the same buses and instructions (memory-mapped), distinguished only by address range; drawback: the CPU cannot fetch an instruction from ROM and access RAM data at the same time.

- **Harvard** — uses separate ports/buses for ROM and RAM (port-mapped), allowing simultaneous instruction fetch and data access, increasing speed; may need different instructions per memory type (Read/Write for ROM, Load/Store for RAM), though high-level compilers like C hide this from the programmer.

### Executing & Modifying Code

- The CPU only reads instructions from ROM via the Program Counter and never writes to it.
- Changing the code in ROM requires an external hardware device such as a Flasher or Programmer connected to the microcontroller.

## 2 Processor Architectures & Performance

### PCs vs. Microcontrollers

- **Harvard** architecture is inherently faster (independent RAM/ROM access, simultaneous fetch and data access), making it ideal for microcontrollers.
- **Von Neumann** is used in PCs because separate buses for data and instructions would create major design and speed issues at scale; PCs boot by loading the OS/code from disk into RAM, after which the processor only interacts with RAM — removing the need for Harvard's complexity.

### Pipelining

- A processor executes instructions in four stages: **Fetch** → **Decode** → **Execute** → **Store**.
- Pipelining overlaps these stages across consecutive clock cycles instead of waiting for one instruction to fully finish, drastically increasing execution speed.
- Harvard architecture supports pipelining easily since instruction fetch and data storage don't conflict.
- **RISC** processors pipeline especially well because most instructions complete in exactly one clock cycle, unlike complex CISC processors.

### What “8-bit Microcontroller” Means

- The Data Bus, ALU, and Registers are all designed to handle 8 bits of data at a time.
- Larger calculations are broken down by the ALU and processed across multiple stages.

### Measuring Speed: MIPS

- System speed is driven by the **Clock**, which generates a steady square wave.
- **MIPS** (Million Instructions Per Second) measures CPU performance — e.g., an 8 MHz RISC microcontroller executing one instruction per cycle runs at 8 MIPS.

### Clock Systems & Oscillators

| Oscillator              | Pros                                                                                | Cons                                                                     |
|-------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| RC (Resistor–Capacitor) | Cheapest option; high immunity to mechanical vibration (e.g., car tires)            | Lowest accuracy, slow startup, vulnerable to temperature changes and EMI |
| Crystal                 | Highest accuracy, fastest startup, excellent immunity to temperature and EMI        | Most expensive; highly sensitive to external mechanical vibration        |
| Ceramic                 | Middle ground on cost, accuracy, and startup time; resists temperature and EMI well | Still negatively affected by physical vibration                          |

## 3 Memory Mapping & Bus Architecture

### Memory Mapping & Addressing

- Peripherals (e.g., GPIO, ADC) are connected using a **Memory Mapped** approach — each device gets its own unique, independent address range.
- To access a register, a programmer finds the peripheral's **Base Address** (start of the peripheral) and the **Offset** (distance to the target register) in the MCU datasheet's Memory Map.

### Master–Slave Architecture

- The CPU (or DMA) is the **Master** — it initiates transactions and commands.
- Peripherals (e.g., GPIO) are **Slaves** that receive and execute commands, communicating over a System Bus.

### Write Transactions

- To send a command (e.g., set a pin High), the CPU builds a pointer to the target register (Base Address + Offset) and initiates a Write Transaction.
- The Master sends three elements across the bus: the **Address**, the **Data**, and the **Size** of the data.

### Design Considerations: Size & Overlapping

- Each peripheral must be allocated enough address space for all its internal registers (e.g., 3 registers of 4 bytes each needs at least 12 bytes).
- Too little allocated space can cause a write to spill over into an adjacent peripheral's address range.
- **Reserved** address gaps are left between peripherals to prevent overlap and to leave room for future features.

### Bus Protocols & Bridges

- Connecting every component to one bus would overload it, so systems use multiple buses with standard protocols such as **AXI**, **AHB**, and **APB**; AXI, for example, suits systems with multiple Masters but is more costly.
- Since different buses use different protocols, a **Bridge** connects them — acting as a Slave to the incoming bus and a Master to the outgoing bus, translating communication between protocols.

## 4 Register Programming & System-Level Design

### Software Programming & Register Access

- **Bit manipulation:** a bitwise OR combined with a bit shift sets a specific bit (e.g., one LED on bit 16) without disturbing the others.
- **Address calculation:** since the compiler doesn't know register names, the absolute address is computed as Base Address + Offset, then cast to a pointer (e.g., `uint32_t*`) so it's treated as a memory location.
- **Structs:** the cleanest approach maps all of a peripheral's registers into a C struct, enabling readable code like `GPIOA->ODR` instead of raw address arithmetic.

### The volatile Keyword

- Essential for hardware registers whose values can change externally (e.g., an input switch) without software intervention.
- Compilers normally optimize by caching a register's value if the software never appears to change it, causing the code to disconnect from real hardware state.
- Declaring a variable **volatile** disables this optimization, forcing a fresh read from the actual memory address every time.

### Bus Frequencies & Power Optimization

- MCU datasheets (e.g., STM32F103) show multiple buses running at different maximum speeds (APB1 up to 36 MHz, APB2 up to 72 MHz).
- Since operating frequency is directly proportional to power consumption, peripherals that don't need high speed are placed on the slower bus to reduce overall power use.

### Master, Slave & DMA

- The **Master** (e.g., CPU) initiates transactions (address, data, size); the **Slave** (memory/peripherals) only responds and cannot initiate on its own.
- For routine bulk transfers (e.g., moving an image to a display), the CPU would waste processing time — a **DMA** unit acts as an independent Master, handling transfers in the background to free up the CPU, while also having a Slave interface to receive configuration commands from the CPU.

### AMBA Standard

- **AMBA** (Advanced Microcontroller Bus Architecture) is an open standard by ARM that unifies communication protocols (AHB, APB, etc.) between internal MCU modules.
- Its key advantage is **reusability**: a peripheral (e.g., an ADC) from one company can be integrated into another company's microcontroller as long as both use the same AMBA protocol.
- Protocols mainly differ in **implementation cost** and **bandwidth** (data transferred at once).